

COMP9414 Artificial Intelligence

Tutorial topic: reasoning with uncertainty with ai9414

UNSW Sydney

1 Introduction

This tutorial practises the main ideas from the reasoning with uncertainty lecture:

- interpreting probabilities as beliefs over possible states;
- using Bayes' rule to update belief from noisy evidence;
- keeping a whole belief distribution instead of one guessed state;
- applying a simple Bayes filter with sensor and motion updates;
- connecting confidence, calibration, and decisions under uncertainty.

The running example is a robot in a one-dimensional corridor. The robot does not know exactly which cell it occupies. It receives noisy observations, such as “I see a door”, and takes uncertain actions, such as “move right”. Its belief is a list of probabilities, one probability per corridor cell.

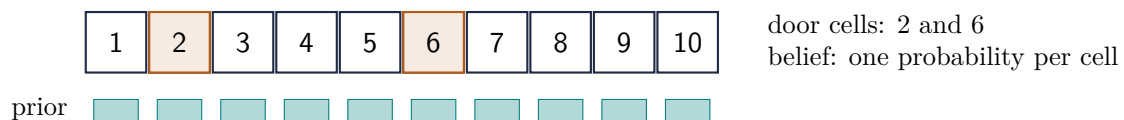


Figure 1: A 10-cell corridor with identical doors at cells 2 and 6. A uniform prior assigns probability 0.1 to each cell.

Vocabulary. A *prior* is the belief before new evidence. A *likelihood* says how probable the evidence is under a possible state. A *posterior* is the updated belief after using the evidence.

2 Installation and setup

2.1 Create a Python environment

The package requires Python 3.11 or newer. On macOS or Linux, create and activate a virtual environment as follows:

```
mkdir ai9414-uncertainty-tutorial
cd ai9414-uncertainty-tutorial
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install ai9414
```

On Windows PowerShell, the activation command is usually:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install ai9414
```

If you are working inside the course repository, the local environment is already available as `.venv`. In that case, commands can be run with:

```
.venv/bin/python -m ai9414 list
```

or, if you use `uv`:

```
uv run ai9414 list
```

2.2 Run the uncertainty demo

Each command starts a local web app and opens a browser window. Leave the command running while you use the demo, then stop it with `Ctrl-C`.

```
ai9414 demo uncertainty
```

Inside the repository, use:

```
uv run ai9414 demo uncertainty
```

To see the available examples:

```
ai9414 list --examples uncertainty
```

The current examples include:

- `two_doors`: a 10-cell corridor with doors at cells 2 and 6;
- `ambiguous_corridor`: repeated landmarks keep belief split;
- `slippery_floor`: uncertain motion spreads belief;
- `bad_sensor`: the sensor is no better than a false alarm model;
- `confident_but_wrong`: a poor prior can produce misleading confidence.

The same demo can be launched from Python:

```
from ai9414.uncertainty import BayesFilterDemo

app = BayesFilterDemo.example("two_doors")
app.show()
```

3 Probability as belief

A probability distribution assigns probability mass across mutually exclusive possibilities. In the corridor problem:

$$\text{bel}(x) = P(\text{robot is in cell } x).$$

For a 5-cell corridor, a uniform prior is:

$$[0.2, 0.2, 0.2, 0.2, 0.2].$$

The values sum to one, so this is a valid belief distribution.

3.1 Task: normalise a distribution

Suppose a sensor update produced these unnormalised values:

$$[0.04, 0.16, 0.04, 0.16, 0.04].$$

1. Compute the normalising constant Z , the sum of the five values.
2. Divide each value by Z .
3. Check that the resulting distribution sums to one.
4. Which cells are now most likely?

Normalisation. If unnormalised values are $u_1, \dots, u_n$, then

$$Z = \sum_i u_i, \quad p_i = \frac{u_i}{Z}.$$

Normalisation keeps the relative proportions but makes the total probability equal to one.

4 Bayes' rule

Bayes' rule connects prior belief, likelihood, and posterior belief:

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}.$$

For a binary hypothesis, the normalised form is often easiest:

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E \mid H)P(H) + P(E \mid \neg H)P(\neg H)}.$$

4.1 Worked example: room occupancy

A robot wants to know whether a room is occupied:

$$\begin{aligned} P(\text{Occupied}) &= 0.1, \\ P(\text{MotionDetected} \mid \text{Occupied}) &= 0.8, \\ P(\text{MotionDetected} \mid \text{NotOccupied}) &= 0.15. \end{aligned}$$

Then:

$$\begin{aligned} P(\text{Occupied} \mid \text{MotionDetected}) &= \frac{0.8 \times 0.1}{0.8 \times 0.1 + 0.15 \times 0.9} \\ &= \frac{0.08}{0.215} \\ &\approx 0.37. \end{aligned}$$

The detector fired, but the room is still not more likely occupied than empty. The prior and false positive rate matter.

4.2 Task: door sensor update

Use the same formula for a door sensor:

$$\begin{aligned}P(\text{DoorCell}) &= 0.2, \\P(\text{SeesDoor} \mid \text{DoorCell}) &= 0.8, \\P(\text{SeesDoor} \mid \text{NotDoorCell}) &= 0.2.\end{aligned}$$

Compute:

$$P(\text{DoorCell} \mid \text{SeesDoor}).$$

Then answer:

1. Why is the posterior not 0.8?
2. What happens to the posterior if $P(\text{DoorCell})$ is changed from 0.2 to 0.05?
3. What happens if the false alarm rate rises from 0.2 to 0.5?

5 Sensor updates in a corridor

In the ai9414 corridor demo, a sensor update multiplies each prior probability by an observation likelihood, then normalises:

$$\text{bel}'(x) \propto P(\text{observation} \mid x)\text{bel}(x).$$

For a 5-cell corridor with door cells 2 and 4:

$$\text{bel} = [0.2, 0.2, 0.2, 0.2, 0.2].$$

Sensor model:

$$\begin{aligned}P(\text{sees door} \mid \text{door cell}) &= 0.8, \\P(\text{sees door} \mid \text{non-door cell}) &= 0.2.\end{aligned}$$

If the robot sees a door:

Cell	1	2	3	4	5
Prior	0.2	0.2	0.2	0.2	0.2
Likelihood	0.2	0.8	0.2	0.8	0.2
Unnormalised	0.04	0.16	0.04	0.16	0.04

$$Z = 0.44, \quad \text{bel}' = [0.091, 0.364, 0.091, 0.364, 0.091].$$

5.1 Task: no-door observation

Using the same 5-cell corridor and the same sensor model, suppose the robot observes *no door* instead.

1. What is the likelihood for a door cell?
2. What is the likelihood for a non-door cell?
3. Compute the unnormalised values.
4. Normalise the result.
5. Which cells are now most likely?

6 Motion updates

A motion update predicts where the robot may be after an action. In the default right-move model:

Outcome	Probability
move one cell right	0.8
stay in the same cell	0.1
move two cells right	0.1

At the end of the corridor, movement is clamped to the last cell.
If the current belief is:

$$[0.091, 0.364, 0.091, 0.364, 0.091],$$

then after a noisy right move, the predicted belief is approximately:

$$[0.009, 0.109, 0.309, 0.145, 0.427].$$

6.1 Task: trace the probability mass

Use the right-move model above. Starting from the posterior after seeing a door:

$$[0.091, 0.364, 0.091, 0.364, 0.091],$$

answer:

1. How much probability mass from cell 2 moves to cell 3?
2. How much probability mass from cell 2 stays in cell 2?
3. How much probability mass from cell 2 overshoots to cell 4?
4. Which source cells contribute probability mass to cell 5?
5. Why does uncertain motion usually spread belief?

7 A full Bayes-filter cycle

A Bayes filter alternates prediction and correction:

$$\text{prior belief} \xrightarrow{\text{motion update}} \text{predicted belief} \xrightarrow{\text{sensor update}} \text{posterior belief}.$$

For hidden state x_t , action a_t , and observation z_t , the standard form is:

$$\text{bel}_t(x_t) = \eta P(z_t \mid x_t) \sum_{x_{t-1}} P(x_t \mid x_{t-1}, a_t) \text{bel}_{t-1}(x_{t-1}).$$

You do not need to memorise this formula. The practical steps are:

1. shift and spread the old belief using the motion model;
2. multiply by observation likelihoods;
3. normalise.

7.1 Task: run the two-door demo

Run:

```
ai9414 demo uncertainty
```

Select or load the `two_doors` example if it is not already selected. Step through the scripted events:

1. Sense door.
2. Move right.
3. Sense no door.

Record:

1. the most likely cells after the first sensor update;
2. the most likely cells after the motion update;
3. the most likely cell after the final no-door update;
4. one sentence explaining why the first observation is ambiguous;
5. one sentence explaining how motion and later evidence reduce the ambiguity.

8 Using the live Python workflow

The uncertainty demo includes a live Python exercise. The browser can call a local solver that implements the Bayes-filter operations. The supplied starter code asks you to implement:

- `normalise(values)`
- `sensor_update(belief, corridor, sensor, sees_door)`
- `motion_update_right(belief, motion)`

The starter code uses zero-based sets internally. For example:

```
corridor = Corridor(cells=5, door_cells={1, 3})
```

means a 5-cell corridor with doors at student-facing cells 2 and 4.

8.1 Task: implement normalise

Write a function that:

1. computes the total of the input list;
2. raises `ValueError` if the total is zero;
3. returns a new list where each value has been divided by the total.

```
def normalise(values: list[float]) -> list[float]:
    total = ...
    if total == 0:
        ...
    return [...]
```

8.2 Task: implement sensor update

For each cell:

1. decide whether it is a door cell;
2. choose the likelihood for the observed event;
3. multiply prior probability by that likelihood;
4. normalise the list.

Think carefully about the negative observation case. If the robot observes no door, then:

$$P(\text{no door} \mid \text{door cell}) = 1 - \text{hit},$$
$$P(\text{no door} \mid \text{non-door cell}) = 1 - \text{false alarm}.$$

8.3 Task: implement motion update

For a right move, each old cell sends probability mass to up to three destinations:

Destination	Index rule	Probability
same cell	i	<code>motion.stay</code>
one cell right	$\min(i + 1, n - 1)$	<code>motion.success</code>
two cells right	$\min(i + 2, n - 1)$	<code>motion.overshoot</code>

Create a new list filled with zeros, then add each contribution into the appropriate destination cell.

Common mistake. Do not update the belief list in place during the motion update. Use a fresh list. Otherwise, probability mass from earlier source cells can be moved again when you process later cells.

9 Bad sensors and calibration

The lecture also connected uncertainty reasoning to calibration. A confidence score is calibrated if cases assigned probability 0.8 are correct about 80% of the time.

The corridor demo has a `bad_sensor` example. In that example:

$$\text{hit} = 0.45, \quad \text{false alarm} = 0.50.$$

This sensor is not informative for seeing a door: it is slightly more likely to say “door” when the robot is not at a door cell than when it is at a door cell.

9.1 Task: diagnose a bad sensor

Run the `bad_sensor` example and answer:

1. After one positive door reading, do door cells become more or less likely?
2. What does this say about the likelihood model?
3. Why is it dangerous to treat every sensor score as trustworthy?
4. How is this related to calibration in learned classifiers?

9.2 Mini-task: reliability bins

Suppose a classifier makes 10 predictions with confidence between 0.8 and 0.9:

Prediction	1	2	3	4	5	6	7	8	9	10
Correct?	Y	Y	N	Y	N	Y	N	Y	N	Y

1. What is the empirical accuracy in this bin?
2. If the average confidence was 0.85, is the classifier overconfident, underconfident, or roughly calibrated in this bin?
3. What extra information would you want before trusting this conclusion?

10 Your mission

Complete the following tasks during the tutorial.

1. **Environment check.** Use `.venv`, `uv`, or your own virtual environment to run `ai9414 list` and launch `ai9414 demo uncertainty`. Record the title of the default example.
2. **Normalisation.** Complete the normalisation task for `[0.04, 0.16, 0.04, 0.16, 0.04]`.
3. **Bayes' rule.** Complete the door-sensor calculation and explain the role of the prior and false alarm rate.
4. **Sensor update.** Compute the no-door observation update for the 5-cell corridor by hand.
5. **Motion update.** Trace the probability mass from cell 2 and identify all source cells contributing to cell 5.
6. **Demo trace.** Run the `two_doors` example and record the most likely cells after each scripted event.
7. **Live Python.** Implement `normalise`, `sensor_update`, and `motion_update_right` in the downloaded starter code or in a local copy.
8. **Bad sensor.** Run the `bad_sensor` example and explain why the positive reading is not helpful.
9. **Calibration reflection.** In three or four sentences, explain why a confidence score is not automatically a reliable probability.

11 Task checklist

If this was a test or exam, you would be asked to submit:

- the environment check result;
- the normalised distribution and its normalising constant;
- the door-sensor Bayes' rule calculation;
- the no-door sensor update table;
- the motion-update mass tracing answers;

- notes from the `two_doors` demo trace;
- your three live Python function implementations;
- your diagnosis of the `bad_sensor` example;
- the calibration reflection paragraph.

A Formula quick reference

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}$$

$$\text{bel}'(x) \propto P(\text{observation} \mid x)\text{bel}(x)$$

$$\text{bel}'(x') = \sum_x P(x' \mid x, a)\text{bel}(x)$$

B Demo command quick reference

```
ai9414 list
ai9414 list --examples uncertainty

ai9414 demo uncertainty

uv run ai9414 list
uv run ai9414 list --examples uncertainty
uv run ai9414 demo uncertainty
```

C Python quick reference

```
from ai9414.uncertainty import BayesFilterDemo

app = BayesFilterDemo.example("two_doors")
app.show()
```

Custom corridor:

```
from ai9414.uncertainty import BayesFilterDemo

app = BayesFilterDemo(
    cells=10,
    landmarks={"door": [2, 6]},
    initial_belief="uniform",
)
app.show()
```

References

- [1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- [2] Harvard CS50AI. *Week 2: Uncertainty*. <https://cs50.harvard.edu/ai/weeks/2/>.

- [3] UC Berkeley CS188. *Bayes Nets and Decision Networks* lecture material.
- [4] University of Washington CSE571. *Probability and Bayes Filters* lecture material.
- [5] scikit-learn. *Probability calibration*.
<https://scikit-learn.org/stable/modules/calibration.html>.
- [6] Oliver Obst. *ai9414: Interactive teaching demos for artificial intelligence*.
<https://pypi.org/project/ai9414/>.